

- Enterprise Architect Thinking Guide — Part 3
 - Question 6: What's the Strategy for Hot Support / Post-Deployment Support?
 - The EA Mindset
 - Hypercare Support Model — Three Phases
 - Phase 1: War Room (First 72 Hours Post-Cutover)
 - Phase 2: Elevated Support (Week 1-2)
 - Phase 3: Stabilization (Week 3-4)
 - Hypercare Exit Criteria
 - Knowledge Transfer Checklist
 - How I'd Answer in an Interview
 - Question 7: RFP/SOW Stage — Your Input and Disagreements
 - The EA Mindset
 - EA's Role in the RFP Process
 - How I Shape the Technical Solution in an RFP
 - Common Disagreements and How I Handle Them
 - Disagreement 1: "We can do this in 3 months" (Sales/Delivery Pressure on Timeline)
 - Disagreement 2: "We don't need a landing zone — just start migrating" (Scope Reduction to Win on Price)
 - Disagreement 3: "The client says they want lift-and-shift for everything" (Client Pressure)
 - Disagreement 4: "We can do this with 3 engineers" (Understaffing)
 - Red Flags in RFPs That I Raise
 - SOW Review Checklist (What I Verify Before Sign-Off)
 - How I'd Answer in an Interview
 - Summary: 7 Enterprise Architect Principles

Enterprise Architect Thinking Guide — Part 3

Focus: Post-Deployment Support, RFP/SOW Strategy

Question 6: What's the Strategy for Hot Support / Post-Deployment Support?

The EA Mindset

"Migration doesn't end at cutover. The 30 days after go-live are when the real test happens — production traffic, real users, real data volumes. A strong hypercare plan is the difference between 'successful migration' and 'we had to roll back on Monday morning.'"

Hypercare Support Model — Three Phases

Cutover → Phase 1: War Room (72 hours) → Phase 2: Elevated Support (Week 1–2) → Phase 3: Steady State (Week 3–4) → BAU Handover

Phase 1: War Room (First 72 Hours Post-Cutover)

Structure:

```
War Room (physical or virtual bridge – always open)
├── Incident Commander: EA or SRE Lead
│   └── Single point of decision-making
├── Cloud Infrastructure Engineer (24x7)
│   └── Monitors: EC2, EKS, RDS, networking, storage
├── Application Engineer (on-call)
│   └── Monitors: Application logs, error rates, user flows
├── DBA (on-call)
│   └── Monitors: Database performance, replication lag, connection pools
├── Network Engineer (on-call)
│   └── Monitors: VPN/DC throughput, DNS resolution, latency
└── Business / App Owner (available on phone)
    └── Decision authority for rollback or business workarounds
```

What We Monitor (First 72 Hours):

Metric	Threshold	Action if Breached
Application error rate	> 1% (or > 2x pre-migration baseline)	Triage immediately
API response time (P99)	> 2x pre-migration baseline	Investigate, consider scaling
Database CPU utilization	> 80% sustained	Right-size or add read replicas
Database connection count	> 80% of max	Investigate connection leaks
Failed user logins	> 5% increase	Check authentication config
Data consistency	Any discrepancy in reconciliation	Stop and investigate
CloudWatch alarm count	Any new alarms firing	Triage each alarm

Communication Cadence:

- **Hours 0-12:** Status update every 2 hours to stakeholders
- **Hours 12-24:** Status update every 4 hours
- **Hours 24-72:** Status update every 8 hours (morning + evening)

Phase 2: Elevated Support (Week 1-2)

Structure:

- War room closes, normal on-call rotation begins
- Dedicated migration support engineer available during business hours
- Daily standup (15 min) — any issues from previous 24 hours?
- Business users actively validating all functions

Issue Tracking:

Priority	Definition	Response SLA	Resolution SLA
P1 — Critical	Service down, revenue impact	15 minutes	4 hours
P2 — High	Service degraded, workaround exists	30 minutes	8 hours
P3 — Medium	Functionality issue, no business impact	4 hours	2 business days
P4 — Low	Cosmetic, enhancement request	Next business day	Next sprint

Weekly Activities:

- Performance comparison: cloud vs. on-prem baseline (response times, throughput)
- Cost review: actual cloud spend vs. TCO estimate
- Issue log review: are we seeing patterns?
- Knowledge transfer sessions to operations team

Phase 3: Stabilization (Week 3-4)

Transition to BAU (Business As Usual):

Activity	Owner	Deliverable
Operations runbook finalized	Cloud Team → Ops Team	Documented procedures for all operational scenarios
Monitoring handover	SRE → Ops Team	Dashboard walkthrough, alarm ownership transfer
On-call rotation transferred	Migration Team → Ops/SRE Team	Updated PagerDuty rotation
Knowledge transfer complete	Migration Team → Ops Team	Recorded sessions, Q&A documentation
On-prem decommission plan	Infrastructure Team	Timeline for shutting down source servers

Activity	Owner	Deliverable
Post-migration review	EA + All Teams	Lessons learned, migration playbook updates
Hypercare exit sign-off	App Owner + EA	Formal acknowledgment that migration is stable

Hypercare Exit Criteria

All must be met before exiting hypercare:

#	Criteria	Measured By
1	Zero P1 incidents in last 7 days	Incident log
2	Performance within 10% of baseline or better	CloudWatch metrics
3	All P2 incidents resolved or have approved timeline	Issue tracker
4	Business owner sign-off on functionality	Written sign-off
5	Runbooks and documentation complete	Reviewed by ops team
6	On-call rotation transferred to BAU team	PagerDuty config
7	Cloud costs within 15% of estimate	Cost Explorer
8	Backup and DR procedures tested	Test results

Knowledge Transfer Checklist

#	Topic	Delivered To	Format
1	Architecture overview and design decisions	Ops + Dev teams	Recorded presentation + ADR documents
2	Infrastructure walkthrough (Terraform modules, networking)	Cloud Ops	Live walkthrough + documentation
3	Deployment procedures (CI/CD pipeline)	DevOps team	Hands-on session + runbook

#	Topic	Delivered To	Format
4	Monitoring and alerting (dashboards, alarms, escalation)	SRE/Ops team	Dashboard walkthrough + alarm playbook
5	Security controls (IAM roles, KMS, network policies)	Security team	Security architecture document
6	Troubleshooting common issues	Ops team	Runbook with scenarios
7	Cost management (FinOps dashboard, optimization levers)	Finance + Ops	Monthly review process
8	Vendor contacts (AWS Support, TAM, account team)	Ops Lead	Contact sheet

How I'd Answer in an Interview

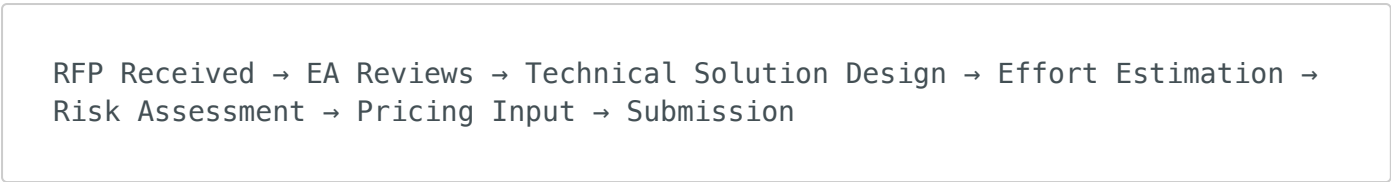
"Post-deployment support follows a three-phase model: 72-hour war room with 24x7 coverage, 2-week elevated support with daily standups, then controlled handover to BAU operations. I define clear exit criteria — zero P1s for 7 days, performance within baseline, business sign-off — so we don't exit hypercare prematurely. The most overlooked part is knowledge transfer — I mandate recorded sessions and reviewed runbooks so the operations team isn't dependent on the migration team's tribal knowledge."

Question 7: RFP/SOW Stage — Your Input and Disagreements

The EA Mindset

"An EA at the RFP/SOW stage is the guardian of technical feasibility and delivery reality. My job is to ensure we're not making promises we can't keep, while also bringing creative solutions that win the deal."

EA's Role in the RFP Process



What I Contribute at Each Stage:

Stage	My Input	Deliverable
Requirements Analysis	Parse RFP for technical requirements, constraints, evaluation criteria	Requirements mapping document
Solution Design	Propose cloud architecture that meets requirements	Architecture diagrams, service selection
Effort Estimation	Bottom-up estimation of engineering effort (not top-down guessing)	Effort breakdown by phase and skill
Risk Assessment	Identify technical risks, assumptions, constraints	Risk register with mitigations
Differentiators	Propose value-adds that competitors won't offer	Innovation proposals (automation, cost savings)
Pricing Review	Validate that margin allows for quality delivery	Red flag if pricing is too aggressive

How I Shape the Technical Solution in an RFP

Example: Client asks for "cloud migration of 500 servers"

What I contribute beyond the obvious:

- 1. **Reframe the problem:** "It's not 500 server migrations — it's X applications that need to work in cloud. Let's think application-centric, not server-centric."
- 2. **Propose the 6 R's assessment:** Show the client we won't just lift-and-shift everything — we'll right-size the approach:

500 servers → Assessment likely shows:

- 100 servers: Retire (20%) — immediate cost saving
- 200 servers: Rehost (40%) — fast, low risk
- 150 servers: Replatform (30%) — managed services
- 50 servers: Refactor (10%) — modernize for agility

3. Include what competitors forget:

- Landing zone setup (multi-account, security baseline)
- Networking (VPN/DC, DNS, firewall rules — these cause delays)
- Cloud cost optimization (right-sizing, Savings Plans)
- Operational readiness (monitoring, runbooks, on-call)
- Hypercare (30-day post-migration support)

4. **Propose phased delivery:** Not a big bang — wave-based with early value demonstration

Common Disagreements and How I Handle Them

Disagreement 1: "We can do this in 3 months" (Sales/Delivery Pressure on Timeline)

What's happening: Sales wants an aggressive timeline to win the deal.

My response:

"A 3-month timeline for 500 servers assumes everything goes perfectly — no dependency delays, no discovery surprises, no business calendar constraints. In my experience, realistic timeline is 6-9 months for quality delivery. I'm happy to propose a 3-month first wave (100 servers) with a phased plan, which gives the client early value while we maintain quality."

How I handle it:

- Present a **risk-adjusted timeline** with explicit assumptions
- Propose **phased delivery** — client gets value early, we don't over-promise
- Document assumptions in SOW: "Timeline assumes network connectivity established by Week 4, client UAT resources available as scheduled, no scope

changes"

- If overruled, document the risk in the SOW annex — "timeline assumes the following..." (CYA)

Disagreement 2: "We don't need a landing zone — just start migrating" (Scope Reduction to Win on Price)

What's happening: Bid team wants to reduce scope to lower price.

My response:

"Skipping the landing zone is like building a house without a foundation. We'll migrate 100 servers into a default VPC with no security baseline, no monitoring, and no cost governance. Then we'll spend 3 months fixing it — at 2x the cost. The landing zone is 3 weeks of work that saves 3 months of rework."

How I handle it:

- Show examples of failed migrations that skipped foundation work
- Calculate the cost of rework vs. doing it right
- Position the landing zone as a **competitive differentiator** — "Our competitors won't include this. We will."
- If overruled, include landing zone as a "recommended Phase 0" — make it a separate line item

Disagreement 3: "The client says they want lift-and-shift for everything" (Client Pressure)

What's happening: Client wants speed and lowest cost. But pure lift-and-shift often costs MORE in cloud than on-prem because of over-provisioned servers.

My response:

"I understand the desire for speed. Let me show you: if we lift these 500 servers as-is, your monthly cloud bill will be X. If we right-size based on actual utilization data (which we collect in the first 2 weeks), the bill drops to 0.6X — that's a 40% savings from Day 1. The right-sizing effort adds 1 week per wave but saves \$Y per month forever."

How I handle it:

- Present **TCO comparison** with and without right-sizing
- Propose: "Lift-and-shift first, right-size immediately after — don't delay migration but optimize within the first month"
- Position right-sizing as part of the service, not an add-on

Disagreement 4: "We can do this with 3 engineers" (Understaffing)

What's happening: Delivery wants to minimize staffing to improve margins.

My response:

"3 engineers can handle 1 wave per month. For 500 servers, that's 10+ months of migration alone, plus the foundation work. If we want to deliver in 6 months, we need 5-6 engineers with cross-functional coverage: cloud infra, database, application, network. I can optimize by using automation — Terraform modules and migration scripts — to increase throughput per engineer, but not below 5."

How I handle it:

- Bottom-up estimation with task-level detail (not gut feel)
- Show the math: hours per server × servers per wave × number of waves
- Propose automation to improve productivity (invest in Terraform templates upfront)
- If overruled, document delivery risk in the SOW

Red Flags in RFPs That I Raise

Red Flag	What It Means	My Recommendation
"Fixed price for migration of all servers"	Client wants cost certainty but scope is undefined	Propose T&M for assessment phase, fixed price for execution after scope is clear
"No downtime during migration"	Unrealistic for large databases	Clarify: "Near-zero downtime using DMS CDC with a 15-minute cutover window"
"Complete in 3 months" with undefined scope	Recipe for failure	Counter with phased approach, or make timeline contingent on assessment findings

Red Flag	What It Means	My Recommendation
No mention of testing/UAT	Client expects us to test everything	Clarify UAT responsibility in RACI — business owns functional testing
"Including all future changes"	Infinite scope creep	Define change management process with clear CR (Change Request) pricing

SOW Review Checklist (What I Verify Before Sign-Off)

#	Item	Why It Matters
1	Scope boundaries clearly defined	Prevents scope creep disputes
2	Assumptions listed explicitly	Protects us when assumptions are wrong
3	Exclusions documented	Client can't claim "I thought that was included"
4	RACI matrix for all activities	Clear accountability — who does testing, who does cutover
5	Change request process defined	How are out-of-scope items handled and priced
6	Acceptance criteria measurable	"Migration successful" means what exactly?
7	Timeline tied to assumptions	"Timeline assumes client provides VPN by Week 4"
8	Staffing model realistic	Can we actually deliver with the proposed team?
9	Risk register included or referenced	Shared risk ownership, not just ours
10	Hypercare / warranty period capped	30 days, not indefinite support

How I'd Answer in an Interview

"At the RFP/SOW stage, my role is threefold: shape the technical solution, validate the commercial feasibility, and protect delivery quality. I bring architecture expertise to design solutions that are technically sound and differentiated. Where I push back is on unrealistic timelines, understaffing, or scope ambiguity — I'd rather have an honest conversation during sales than discover problems during delivery. My approach is to present alternatives: 'We can do 3 months if we phase it, or 6 months if you want everything at once. Here's the risk profile of each.' I always document assumptions, exclusions, and dependencies in the SOW — these are the clauses that save us during delivery."

Summary: 7 Enterprise Architect Principles

Carry these into every interview answer:

#	Principle	Application
1	Decisions are data-driven, not opinion-driven	Use scoring matrices, TCO models, benchmark data
2	Think business outcome, not technology	"This saves \$11M over 3 years" beats "AWS has more services"
3	Plan for failure	Risk registers, rollback plans, hypercare — always have Plan B
4	Dependencies are managed, not ignored	RAID logs, early ordering, escalation paths
5	Security is a design constraint, not an add-on	Bake security into modules, landing zones, CI/CD
6	Communicate impact, not details	"This delay costs \$200K/month" moves stakeholders faster than "Direct Connect provisioning is taking longer"

#	Principle	Application
7	Document everything	Assumptions in SOW, decisions in ADRs, incidents in post-mortems — protect yourself and the project

You're not just answering questions in the interview — you're demonstrating that you THINK like a leader who's done this at scale. Every answer should show: "I've seen this before, here's how I handled it, and here's how I'd do it even better next time."

Good luck, Pushparaj! 🚀